IST 387: Introduction to Applied Data Science
Lab 11: Notes

Notes Prepared by: Dr. Preeti Jagadev

Reference: Data Science for Business with R, written by Jeffrey S. Saltz and Jeffrey M. Stanton

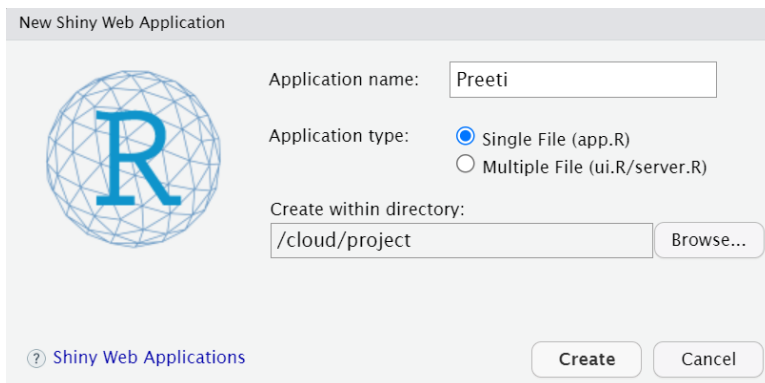
Contents of the Notes:

- Creating Web Applications in R
- Deploying the Application
- Important R Functions

CREATING WEB APPLICATIONS IN R

Sometimes, when we use R to solve a problem, it is helpful to create an interactive application, especially one that can be used from within a web browser. Fortunately, we can create interactive web applications in R using the **Shiny package** created by the folks at RStudio. Let's get started!

- The first step, is to click on the File menu,
- Then New File,
- Then create a "**Shiny Web App**."
- When you see this dialog window, press the create button.
- After clicking create, select single file as the application type.



- An R file (app.r) will be created.
- This code will be visible as a tab in your R window.

```
# This is a Shiny web application. You can run the application by clicking
# the 'Run App' button above.
#
# Find out more about building applications with Shiny here:
#
#    https://shiny.posit.co/
#

library(shiny)

# Define UI for application that draws a histogram
ui <- fluidPage(

    # Application title
    titlePanel("Old Faithful Geyser Data"),

    # Sidebar with a slider input for number of bins
    sidebarLayout(
        sidebarPanel(
            sliderInput("bins",
                        "Number of bins:",
                        min = 1,
                        max = 50,
                        value = 30)
        ),

        # Show a plot of the generated distribution
        mainPanel(
           plotOutput("distPlot")
        )
    )
)
```

```
# Define server logic required to draw a histogram
server <- function(input, output) {

    output$distPlot <- renderPlot({
        # generate bins based on input$bins from ui.R
        x    <- faithful[, 2]
        bins <- seq(min(x), max(x), length.out = input$bins + 1)

        # draw the histogram with the specified number of bins
        hist(x, breaks = bins, col = 'darkgray', border = 'white',
             xlab = 'Waiting time to next eruption (in mins)',
             main = 'Histogram of waiting times')
    })
}

# Run the application
shinyApp(ui = ui, server = server)
```

Wow, a lot of code was created! Even though it looks complex, the Shiny package has done us a huge favor by creating a basic interactive application that already works. There are three main components of the code:

1. User interface (ui).
2. The code that does the R calculations (the server).
3. Code that connects the ui and the server by calling shinyApp( ) to get the application started.

- As we can see, the code creates the user interface (this is the code that lets a user adjust the application, such as selecting an attribute). That code defines an object called "**ui.**"

- Then, the code runs on the server (think of this as the code that performs the functions needed to display the appropriate information on a web page). That code defines an object called "**server**."

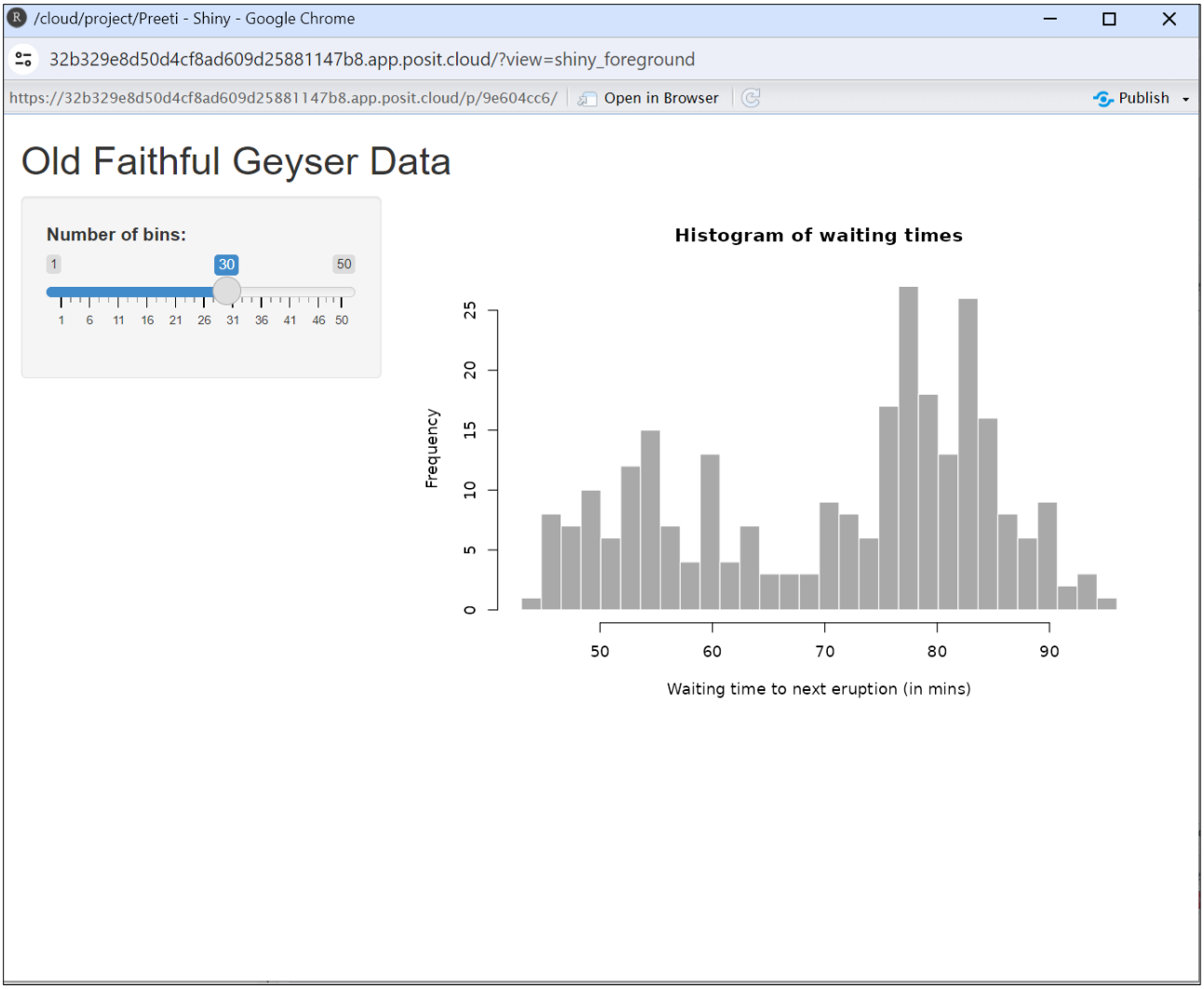
- Finally, one line of code gets the whole process started by identifying the ui and the server objects in a call to **shinyApp( ).** It is important for this **shinyApp( )** function call to be the last line in the file.

We can rename "ui" and "server" with any names we want to use. The call to **shinyApp( )** lets R know the names of these two key components.

For example, if we changed **ui** to **myUI**, the shinyApp would be changed to:

```
shinyApp(ui = myUI, server = server)
```

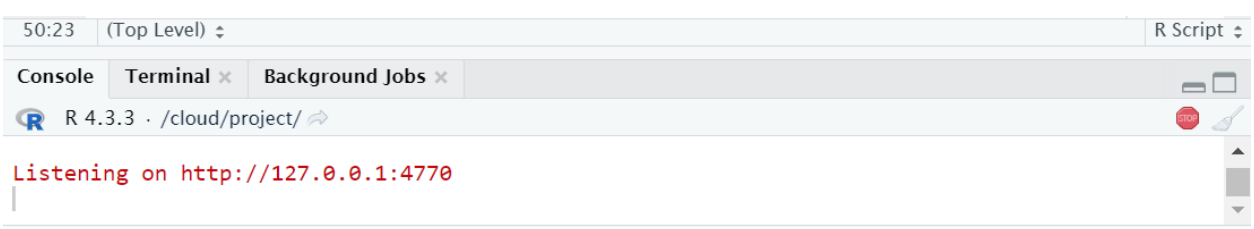
Note that when you are examining or editing the Shiny application file in RStudio, the normal Run button does not appear. Instead, you can now see the "**Run App**" button. Since a working file of code has already been generated, you can press the Run App button and the code will run.



R studio will launch the app in a new window (as shown above), or in the viewer window (the lower-right window in RStudio).

You can choose the app launch in one of these by clicking on the small down arrow icon just to the right of the **Run App button.** RStudio will give you a pull-down menu to select how you want to display the app.

Note that the output on the console says:



This shows that RStudio has invisibly created a temporary, local server on your computer. The "ui" portion of your app that appears in the web page will connect with this temporary server to do all the behind-the-scenes work to show the histogram.

With our application running, we can change the number of bins used to create a histogram, and then the picture adjusts. One important note is that while this app is running in your web browser, it is not yet a "public" web application that other people can use (we will get to that late). For now, it is an interactive application, running locally on your machine.

Occasionally, after closing the application window (e.g., the window tab in a web browser where the R code was running), RStudio might not terminate the server application. So, if you **hit enter** in the **console of RStudio**, you might not see the > prompt. If this is the case, then it is likely the case that the temporary R server is still running. If this happens, just click the Stop button (in the upper-right part of the console window). This will cause posit cloud to shut down the server and return control to the R console command prompt.

Now that we got the code to run and then stop, let's explore the code in more detail:

- First, within the ui, which is defined by a function called **fluidPage( ),** we can see there are two areas defined: **titlePanel,** which is just a space to put the title of the app, and **sidebarLayout( ),** which is the space where all the activity happens.
- Within **sidebarLayout**, there is **sidebarPanel( ),** which contains one interactive control called **sliderInput( )**. This control defines the number of bins to be used. The **sidebarLayout( )** also contains **mainPanel( )** where we plot (draw) the histogram.
- Next, move on to the block of code that begins with "**server**." Note that the function that we are defining for "server" has two arguments: **input** and **output**. In the **ui**, when we called **sliderInput**, we defined a data slot called "**bins**." As a result, the server now can access **input$bins**, to get the information from the slider.
- The server uses that value to help define the number of bins for the histogram in this line of code:

```
bins <- seq(min(x), max(x), length.out = input$bins + 1)
```

Next, looking back at the **ui**, when we called **plotOutput( )** we specified the name "**disPlot.**" As a result, the server code can now access a data object called **output$distPlot**. This data object stores the rendering of the histogram so that it can be shown within the web page. Let's make a couple of small changes to the server code to prove to ourselves how things work:

```
# Define server logic required to draw a histogram
server <- function(input, output) {

    output$distPlot <- renderPlot({
        # generate bins based on input$bins from ui.R
        x    <- faithful[, 2]
        bins <- seq(min(x), max(x), length.out = input$bins + 1)

        # draw the histogram with the specified number of bins
        hist(x, breaks = bins, col = 'darkgray', border = 'white',
             xlab = 'Waiting time to next eruption (in mins)',
             main = 'Histogram of waiting times')
    })
}
```

DEPLOYING THE APPLICATION

Although this Shiny app runs within RStudio/Posit Cloud, it is still not a web application. We still must deploy our application to a public server if we want other people to use it. There are many ways to deploy the app, and one of the easiest is to use the hosting service available at http://www.shinyapps.io/, which is free for testing small applications.

What makes shinyapps.io so nice is that it is integrated with RStudio/Posit Cloud. To get our interactive web page to work, we first need to create an account at shinyapps.io, and then follow instructions in Studio, using the following screens:

STEP 1 – INSTALL RSCONNECT

The `rsconnect` package can be installed directly from CRAN. To make sure you have the latest version run following code in your R console:

```
install.packages('rsconnect')
```

STEP 2 – AUTHORIZE ACCOUNT

The `rsconnect` package must be authorized to your account using a token and secret. To do this, click the copy button below and we'll copy the whole command you need to your clipboard. Paste it into your R console to authorize your account. Once you've entered the command successfully in R, that computer is now authorized to deploy Shiny for R applications to your shinyapps.io account.

```
rsconnect::setAccountInfo(name='cmrywb-preeti-jagadev',
                          token='[redacted]
                          secret='<SECRET>')
```

Show Secret

Copy to clipboard

In the future, you can manage your tokens from the Tokens page the settings menu.

STEP 3 – DEPLOY

Once the `rsconnect` package has been configured, you're ready to deploy your first application. If you haven't written any applications yet, you can also checkout the Getting Started Guide for instructions on how to deploy our demo application. Run the following code in your R console.

```
library(rsconnect)
    rsconnect::deployApp('path/to/your/app')
```

We get the following in the Console:

Console | Terminal | Background Jobs

R 4.3.3 · /cloud/project/

```
> rsconnect::deployApp('Preeti')
── Preparing for deployment ──────────────────────────────────────────────
Found multiple accounts.
Which one do you want to use?
1: server: posit.cloud / username: Preeti Jagadev
2: server: shinyapps.io / username: cmrywb-preeti-jagadev
Selection: 2
✓ Deploying "preeti" using "server: shinyapps.io / username: cmrywb-preeti-jagadev"
ℹ Creating application on server...
✓ Created application with id 11745748
ℹ Bundling 1 file: app.R
ℹ Capturing R dependencies with renv
✓ Found 31 dependencies
✓ Created 18,099b bundle
ℹ Uploading bundle...
✓ Uploaded bundle with id 8487592
```

```
── Deploying to server ──────────────────────────────────────────────────────
Waiting for task: 1405163950
  building: Building image: 10315375
  building: Fetching packages
  building: Installing packages
  building: Installing files
  building: Pushing image: 10315375
  deploying: Starting instances
  rollforward: Activating new instances
  success: Stopping old instances
── Deployment complete ──────────────────────────────────────────────────────
✓ Successfully deployed to <https://cmrywb-preeti-jagadev.shinyapps.io/preeti/>
```

As you can see, we just deployed a public test of a simple R application. We now have a real web-based application. Of course, there are many ways to configure/define input within Shiny—a slider is just one possibility.

Important Functions

- **setAccountInfo( ):** Enables the creation of web-based hosted applications.
- **shinyApp( ):** Runs the shiny application, defining the ui and the server.
- **Run App:** Is a new way to run an application, not just R code.